CyberFlow IDS Final Report

Cover Page

Project: CyberFlow IDS

Author: Dr. Raul C. Gacusan

Program/Course: Post-Graduate Diploma in Artificial Intelligence and Machine Learning

Institution: Asian Institute of Management (AIM)

Date: July 3, 2026

Repository Link: https://github.com/RaulGacusan748/cyberflow_ids

Google Colab Link:
https://colab.research.google.com/github/RaulGacusan748/cyberflow_ids/blob/main/notebooks/cyberflow_ids_tournament.ipynb

---

Abstract

CyberFlow IDS is an intrusion detection workflow that combines cybersecurity traffic preprocessing, supervised model tournament evaluation, and live replay-based inference to classify benign and malicious traffic patterns. This project demonstrates reproducible model selection and practical deployment outputs for both technical and non-technical stakeholders.

---

1. Introduction

Network intrusion detection remains essential as enterprise environments continue to generate high-volume, heterogeneous traffic. Signature-only approaches can miss novel attacks, motivating machine learning models that can generalize from behavioral indicators.

This project proposes an end-to-end IDS pipeline named CyberFlow IDS, designed to:

- preprocess cleaned CIC-IDS-2017 derived data,
- evaluate multiple models under a consistent training protocol,
- select the strongest model via leaderboard metrics,
- support a dashboard and a terminal replay workflow for demonstration.

---

2. Objectives

- Build a reproducible machine learning IDS pipeline.
- Compare candidate classifiers in a tournament framework.
- Validate the selected model on held-out traffic data.
- Demonstrate practical usability through dashboard and replay simulation.

Primary business KPIs for this capstone are: reduction in false-positive SOC alerts, reduction in analyst triage time, and improved incident response speed through low-latency model inference.

Capstone linkage: this Module 1 output maps directly to Capstone Steps 1-3 by establishing the problem framing, task definition, and target success metrics.

---

3. Related Work and Context

Traditional IDS systems include signature-based and anomaly-based methods. Machine learning IDS expands anomaly detection by learning multi-feature traffic behavior and enabling higher adaptability.

The CyberFlow IDS design follows the anomaly-detection direction of contemporary IDS research by combining interpretable preprocessing, leakage-aware feature screening, and tournament-based model selection. Instead of relying on a single classifier, multiple candidate learners are evaluated under a consistent train/validation split so the final deployed model reflects empirical performance rather than assumption. This aligns with current applied cybersecurity ML practice, where model robustness, latency, and false-positive control are prioritized for SOC usability.

---

4. Dataset and Preprocessing

4.1 Dataset Source

- Primary dataset family: CIC-IDS-2017 derivatives.
- Working input file path in this project:
  - data/raw/cicids2017_cleaned.csv

4.2 Preparation Steps

- Data cleaning and label normalization.
- Feature selection / exclusion of unsupported fields.
- Numerical scaling with persistent scaler serialization.
- Train/validation split for tournament evaluation.

Specific preprocessing used in implementation:

- Binary target remapping was applied as `Target = 0` for benign/normal classes and `Target = 1` for attack classes.
- Non-feature metadata columns were removed before training, including: `Label`, `Target`, `Flow ID`, `Source IP`, `Source Port`, `Destination IP`, `Destination Port`, `Timestamp`, and `Protocol` (when present).
- Numeric-only feature projection was enforced for model compatibility.
- Constant features (zero variance) were excluded.
- A leakage shield checked linear correlation against target labels and excluded any near-proxy features above threshold (`|r| > 0.98`).
- Data split used stratified 80/20 partitioning (`random_state=42`) to preserve class distribution.
- `StandardScaler` was fitted on train data and persisted as `models/scaler.joblib` for consistent production inference.

---

5. Methodology

5.1 Model Tournament Strategy

Candidate models are trained under consistent data partitions and compared by performance metrics (e.g., accuracy, precision, recall, F1, ROC-AUC where applicable).

5.2 Winner Selection

A leaderboard ranks candidate models. The top model is serialized as production artifact for downstream inference.

Tournament run results (from captured execution log):

- Decision Tree: Accuracy 0.9921, Precision 0.9889, Recall 0.9643, F1 0.9764, Train Time 1.98s
- Random Forest: Accuracy 0.9943, Precision 0.9945, Recall 0.9716, F1 0.9829, Train Time 1.42s
- XGBoost: Accuracy 0.9956, Precision 0.9910, Recall 0.9826, F1 0.9868, Train Time 2.59s
- Naive Bayes: Accuracy 0.6422, Precision 0.3181, Recall 0.9793, F1 0.4803, Train Time 0.16s

Champion model: **XGBoost** (F1-Score 0.9868), serialized to `models/intrusion_detector.joblib`.

---

6. Implementation

6.1 Training Engine

- Core file: scratch/model_training_tournament.py
- Output artifacts:
  - models/intrusion_detector.joblib
  - models/scaler.joblib

6.2 Dashboard

- Core file: src/dashboard.py
- Purpose: interactive traffic analytics and prediction interface.

6.3 Live Replay Detector

- Core file: scratch/live_detector.py
- Purpose: terminal-based simulation/replay and real-time style classification output.

---

7. Results

7.1 Model Leaderboard Summary

The XGBoost classifier achieved the strongest overall balance between precision and recall, producing the highest F1-Score (0.9868) while maintaining very high accuracy (0.9956). Random Forest ranked second with competitive precision and throughput. Naive Bayes showed high recall but poor precision and low F1, indicating over-flagging behavior that is unsuitable for production SOC alert quality.

7.2 Operational Validation

The live replay engine successfully loaded serialized artifacts and executed packet-level inference under low-latency conditions (single-digit to low double-digit milliseconds in the sampled replay run). This confirms deployment readiness for interactive monitoring workflows.

7.3 Error and Risk Observations

- Replay simulation in this run produced safe classifications across sampled packets, suggesting additional adversarial replay scenarios should be tested for broader stress validation.
- Results are dataset-dependent; cross-dataset testing and temporal drift checks are recommended to confirm generalization.
- Feature and label schema consistency must be preserved between training and production to avoid transformation mismatch.

---

8. Discussion

- Strengths: reproducibility, clear tournament comparison, deployable artifacts.
- Limitations: dataset bias, class imbalance, and environment-specific generalization limits.
- Practical implications for SOC workflows and triage support.

---

9. Conclusion and Future Work

CyberFlow IDS demonstrates an operational machine learning IDS workflow from data preparation to deployable inference components. Future work can include online learning, drift monitoring, and broader benchmark datasets.
